

Introductions to Big Data Systems: Homework 9

Francisco Diego Ariel Romero Contreras

法兰西思科

francisco.romero.c@ug.uchile.cl

2025403471

Introduction

The two selected algorithms for this homework are:

- K-cores.
- Page Rank Delta.

Algorithms

1. K-cores

Given a graph the algorithm of k-cores return the vertex within the k-cores of the graph in the file coreId inside the graph.

This implementation identify if there is any vertex below k, if there were any then this vertex is marked into to_remove Bitmap and all the neighbors vertex degree is reduced by 1. if there isn't any vertex below k, then stops and write the vertex within the k-core graph into a file.

Some important decisions in this implementation are: The active and to_remove BigVector in really are used as Bitmap, this because the Bitmap interface given in GridGraph doesn't support the method reset, which return to 0 only the bit select.

pseudo-code:

```
INPUT Graph G, Integer k
G = (V,E)
active <- fill(1)
degree <- fill(0)
for each v in V add 1 to degree[v]

changed <- true
iter <- 0

while changed
    changed <- false
    iter <- iter + 1
    to_remove <- fill(0)

    for each v in V
        if (active[v] and degree[v] < k)
            to_remove[v] <- 1
            changing = true
    for each e in E matching (s,t)
        if (activate[s] == 0 and activate[t] == 1)// the
            degree[t] <- degree[t] - 1
    for each v in V
        if (to_remove[v] == 1)
            active[v] <- 0

OUTPUT <- for v in V
            if (active[v] == 1 )
                write v
```

2. Page Rank Delta

Given a graph the algorithm of pagerank-delta return the rank of each page using the delta variation in the file RankDelta inside the graph.

The implementation make the calculus for the contribution on each active page, for the first iteration all the page are active, for the rest of iteration is determinate if the threshold is lower than $\frac{\Delta(A)}{\text{Rank}(A)}$.

pseudo-code:

```
Input Graph G, Integer maxIter

G = (V,E)

threshold <- 0.15
damping <- 0.85

active <- fill(1)

for v in V
  pagerank[v] <- 1
  if (degree[v] > 0)
    delta[v] <- 1 / degree[v]
  else
    delta[v] <- 0

for iter in range(0, maxIter)
  next_delta <- fill(0)
  active_next <- fill(0)

  for each edge in E matching (s,t)
    if(active.getbit(s))
      if(degree[s] > 0)
        contrib <- delta[s] / degree[s]
        next_delta[t] <- next_delta[t] + d * contrib

  for each v in V
    if (next_delta[v] != 0)
      pagerank[v] <- pagerank[v] + next_delta[v]
      rel <- next_delta[v] / pagerank[v]
      if (rel > threshold)
        active_next.setbit(v)
        delta[v] = next_delta[v]
  swap active next_active

  active_count <- 0
  for v in V
    if (active[v])
      active_count <- active_count + 1

  if (active_count == 0)
    break
OUTPUT <- for v in V
  if (active[v] == 1 )
    write pagerank[v]
```